

Parallel Programming Mid-Term Project: Morphological Opening with OpenMP

1. Abstract

*This report details the implementation and performance analysis of parallelizing the morphological image processing operation, **Opening** (Erosion followed by Dilation), using the OpenMP framework in C++. The goal was to compare the performance of sequential and parallel implementations across varying thread counts on two distinct image workloads. The parallel version successfully achieved a maximum speedup of **2.67x** (Test Set 1) and **2.46x** (Test Set 2). The analysis demonstrates scaling up to the logical core limit (N=8), but also highlights the impact of parallelization overhead, leading to diminishing efficiency.*

2. Introduction

The project focused on the parallel implementation of the morphological image Opening operation, which is a key process in computer vision and image processing for noise removal and smoothing boundaries.

The main objectives were:

- 1. Implement both sequential and parallel (OpenMP) versions of the Opening (Erosion and Dilation) algorithm.*
- 2. Measure execution time, calculate Speedup, and determine Parallel Efficiency for different thread counts.*
- 3. Analyze the scalability and identify the optimal thread count for the given workloads.*

*The kernel size used for both Erosion and Dilation was **9x9**.*

3. Implementation Details

The application was developed in C++ and utilized OpenMP directives for parallelization.

3.1. Implemented Operation: Morphological Opening (Erosion + Dilation)

*The operation implemented is the **Opening**, a fundamental filter in mathematical morphology. It is defined as an **Erosion** (replaces the center pixel with the minimum value in the kernel window) followed by a **Dilation** (replaces the center pixel with the maximum value). The overall effect is to smooth object contours, remove small isolated noise, and open narrow gaps or bridges.*

3.2. Parallel Implementation (OpenMP)

*The parallelization strategy employed **data parallelism** by dividing the image processing work*

among multiple threads. The core loops for both *Dilate_Parallel* and *Erode_Parallel* were parallelized using the `#pragma omp parallel for` directive with `collapse(2)` to flatten the two outer loops (height *i* and width *j*) into a single, large iteration space, maximizing the workload available for parallel execution.

The specific OpenMP directive applied was:

```
#pragma omp parallel for collapse(2) schedule(static)
```

```
for (int i = 0; i < height; ++i) {
```

```
    for (int j = 0; j < width; ++j) {
```

```
        // ... Kernel iteration for Erode/Dilate operation
```

```
    }
```

```
}
```

- **collapse(2):** Ensures both image dimensions (*i* and *j*) are scheduled together for better load balancing and larger chunks of work.
- **schedule(static):** Provides predictable and low-overhead scheduling, which is generally suitable for dense, uniform computational loops like image convolutions.

4. Experimental Setup

4.1. Hardware Specifications

The performance tests were executed on the following machine configuration (based on the system information provided):

Component	Specification
CPU	Intel(R) Core(TM) i7-8550U @ 1.80GHz
Cores	4 Physical Cores

Logical Processors	8 Threads (using Hyper-Threading)
RAM	16.0 GB DDR4

4.2. Workloads and Datasets

The performance was tested on two image sets with significantly different characteristics, enabling a robust analysis of scalability and Amdahl's Law:

1. **Test Set 1 (High-Volume):** Used the **Indian Diabetic Retinopathy Image Dataset figure1 [1]**. These images are characterized by high resolution and complex structures, resulting in a large processing workload.
2. **Test Set 2 (Low-Volume):** Used the **Satellite Images of Water Bodies Dataset figure3 [2]**. This dataset features a lower overall processing requirement because most of the images have a smaller size than the first dataset, which is more sensitive to parallel overhead costs.

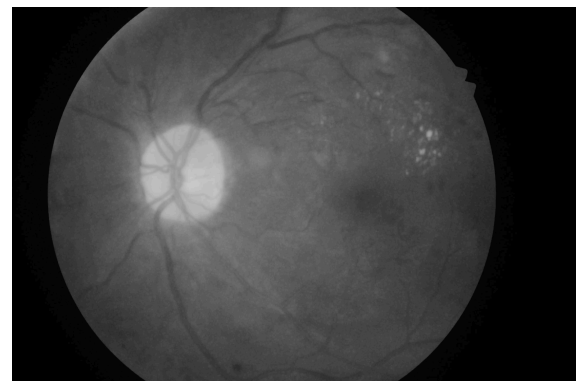


FIGURE 1: images from indian diabetic retinopathy image dataset and results after morphology

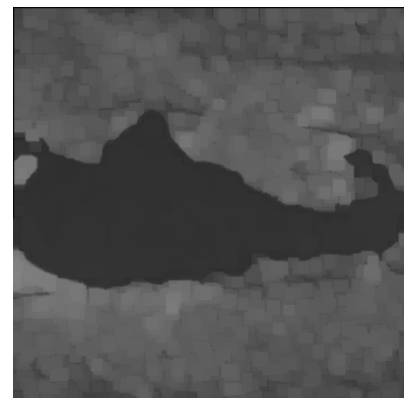
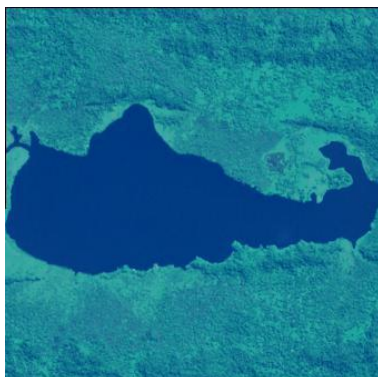


FIGURE 2: images from Satellite Images of Water Bodies Dataset and results after morphology

4.3. Metrics

- **Sequential Time (T_s):** Execution time of the sequential version (used as the baseline).
- **Parallel Time (T_p):** Execution time of the parallel version.
- **Speedup (S):** Calculated as $S = T_s / T_p$.
- **Efficiency (E):** Calculated as $E = S / P$, where P is the number of threads.

5. Results and Analysis

5.1. Test Set 1: High-Volume Workload

The baseline sequential time for this test set was $T_s = 46,325$ ms (46.325 seconds).

Threads (P)	Parallel Time (ms)	Speedup (S)	Efficiency (E)
1	47880	0.9675	0.9675
2	30939	1.4973	0.7487
4	22408	2.0673	0.5168
8	17313	2.6757	0.3345

Analysis

Speedup increases consistently up to 2.67x at the maximum tested thread count of 8. The parallel time shows clear reductions, but the efficiency decreases as threads increase, reflecting the diminishing returns of Hyper-Threading and the growing cost of parallel overhead relative to the sequential portion of the code. Figure3, Figure4

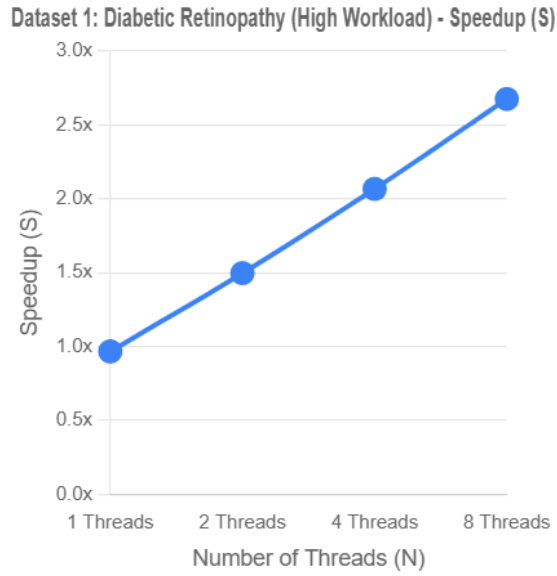


FIGURE 3: dataset 1, speedup vs n. of threads comparison

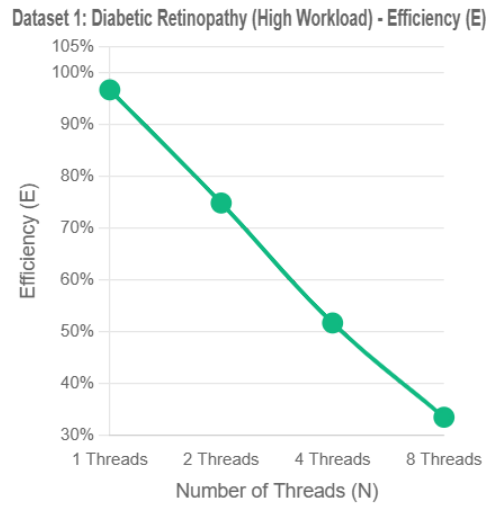


FIGURE 4: dataset 1, efficiency vs number of threads comparison

5.2. Test Set 2: Low-Volume Workload

The baseline sequential time for this test set was $T_s = 2,776$ ms (2.776 seconds).

Threads (P)	Parallel Time (ms)	Speedup (S)	Efficiency (E)

1	2920	0.9507	0.9507
2	1959	1.4170	0.7085
4	1391	1.9957	0.4989
8	1126	2.4654	0.3082

Analysis

Maximum Speedup is 2.46x at 8 threads. The efficiency drops below 50% after 4 threads, which is a key indicator of **Amdahl's Law**. The overhead cost, while fixed, consumes a much larger relative portion of the overall execution time in this smaller workload, leading to a poorer scaling profile compared to the high-volume set. Figure5, Figure6

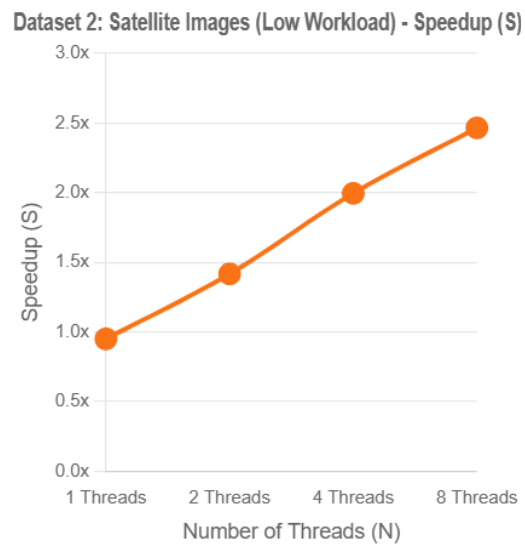


FIGURE 5: dataset 2, speedup vs n. of threads comparison

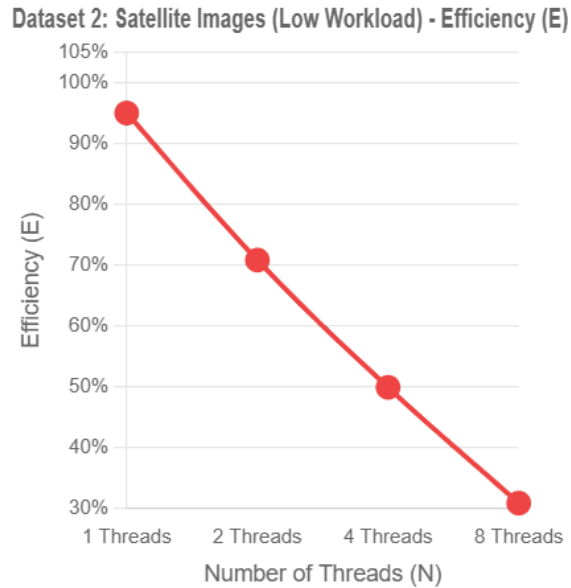


FIGURE 6: dataset 2, efficiency vs number of threads comparison

5.3. Comparative Performance Analysis (Amdahl's Law)

The following figures compare the scaling behavior of both workloads directly, highlighting the impact of problem size relative to parallel overhead. Figure7, Figure8

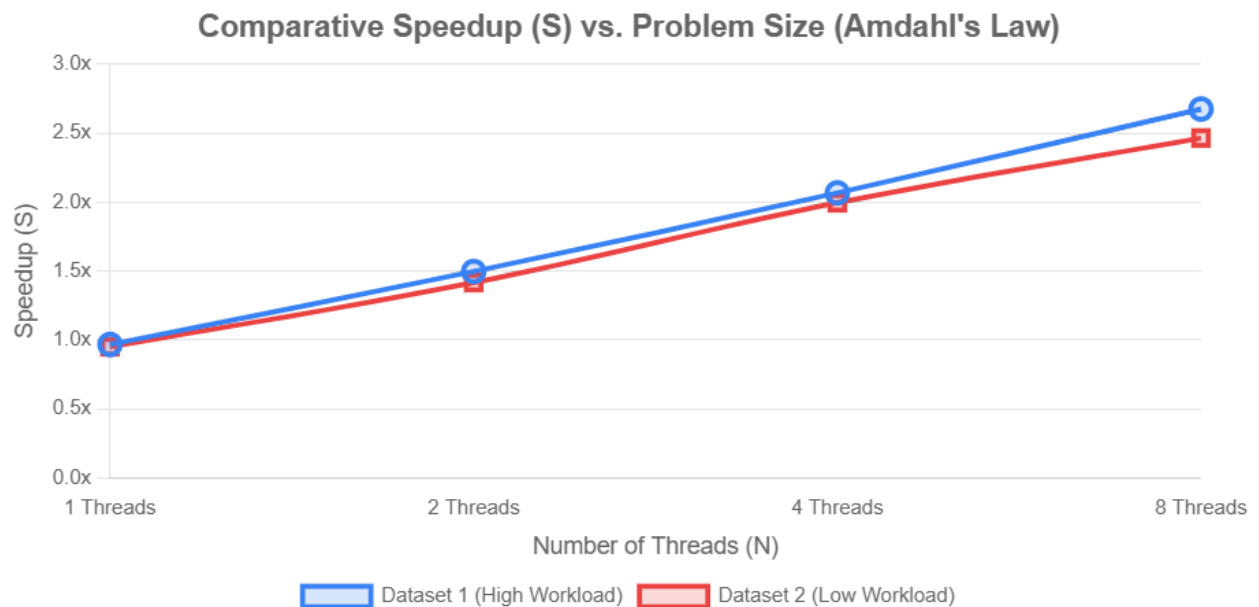


FIGURE 7: Comparison of both dataset → speedup vs n. of threads.

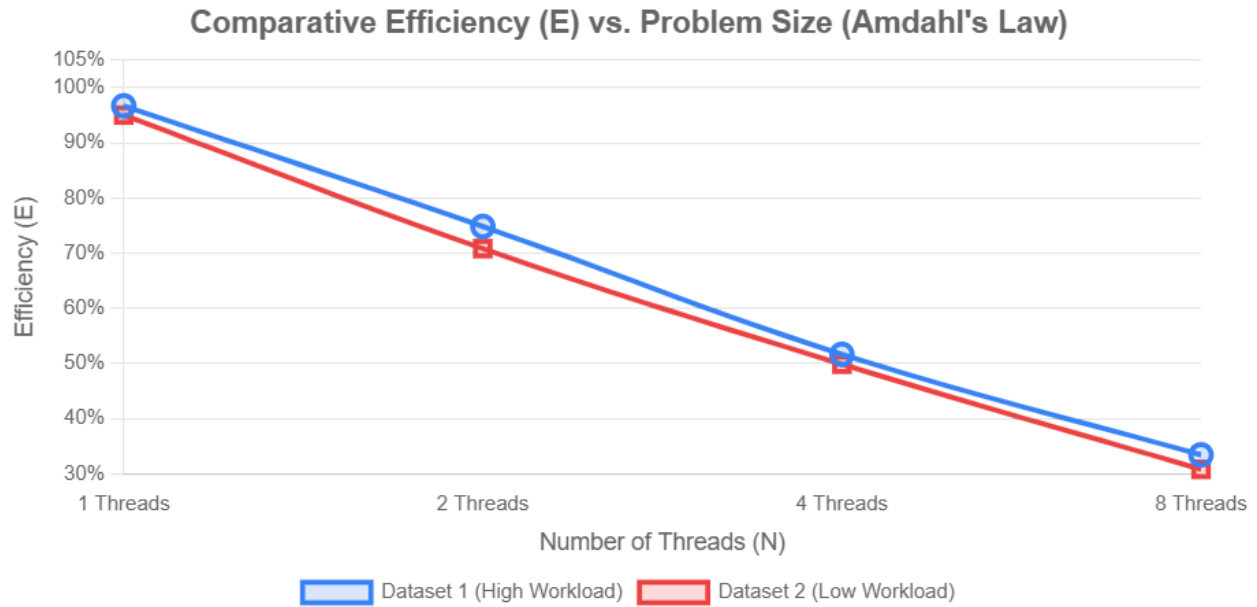


FIGURE 6: Comparison of both datasets → efficiency vs number of threads.

6. Conclusion

The parallel OpenMP implementation of the morphological opening operation demonstrated successful speedup across both tested workloads, with a maximum thread count of 8.

Workload	Sequential Time	Max Speedup	Optimal Speedup Point	Efficiency at Optimal Point
Test Set 1 (High-Volume)	46.325 s	2.67x	8 Threads	33.45%
Test Set 2 (Low-Volume)	2.776 s	2.46x	8 Threads	30.82%

The results confirm the principle that parallelization is most effective when the computational workload is large enough (Test Set 1) to sufficiently mask the inevitable parallelization overhead. Both tests scaled best up to the logical core limit of 8 threads.

8. References

[1] AaryaPatel98. **Indian Diabetic Retinopathy Image Dataset**. Kaggle Dataset Repository URL.

<https://www.kaggle.com/datasets/aaryapatel98/indian-diabetic-retinopathy-image-dataset/code/discussion>

[2] Francisco Escobar. **Satellite Images of Water Bodies**. Kaggle Dataset Repository URL.

<https://www.kaggle.com/datasets/franciscoescobar/satellite-images-of-water-bodies/discussion?sort=hotness>

[3] **Parallel Programming Course Lecture Notes and Slides**. Course material from the Parallel Programming course, [Semester/Year].